

Apache Kafka: Complete Developer Guide

Table of Contents

1. [Introduction](#)
2. [Kafka Fundamentals](#)
3. [Core Concepts](#)
4. [Architecture](#)
5. [Cluster Setup](#)
6. [Developer Setup & Configuration](#)
7. [Producer API](#)
8. [Consumer API](#)
9. [Advanced Topics](#)
10. [Best Practices](#)
11. [Troubleshooting](#)

Introduction

Apache Kafka is a distributed event streaming platform capable of handling trillions of events a day. It is designed to be:

- **Fast:** Latency as low as milliseconds
- **Scalable:** Horizontal scaling across multiple nodes
- **Durable:** Fault-tolerant with replication
- **Distributed:** Decentralized architecture

Use Cases

- Real-time analytics
- Log aggregation
- Stream processing
- Microservices communication
- Event sourcing
- Change data capture (CDC)



Kafka vs Traditional Message Queues

Feature	Kafka	Traditional MQ
Throughput	1M+ msgs/sec	Lower
Durability	Persistent, Replicated	Often in-memory
Scalability	Horizontal	Vertical
Retention	Configurable	Auto-delete
Replay	Full replay capability	Limited

Kafka Fundamentals

What is Kafka?

Kafka is a publish-subscribe messaging system that stores streams of records in categories called **topics**. It is designed as a distributed system with:

- **Producers:** Applications that publish records to topics
- **Consumers:** Applications that subscribe to topics and process records
- **Brokers:** Servers that store data and serve requests
- **Topics:** Named feeds of records
- **Partitions:** Divisions of topics for parallelism

Key Characteristics

1. **Distributed:** Runs on clusters of machines
2. **Fault-tolerant:** Handles node failures gracefully
3. **High-throughput:** Optimized for processing millions of messages
4. **Low-latency:** Near real-time processing
5. **Durable:** Persists data to disk with replication



Core Concepts

1. Topics

A topic is a named channel to which records are published. Think of it as a stream or feed of data.

Topic: user-events

```
|— Partition 0 [0][1][2][3]...  
|— Partition 1 [0][1][2][3]...  
|— Partition 2 [0][1][2][3]...
```

Properties:

- Name: Unique identifier
- Partitions: Number of parallel streams
- Replication Factor: Number of copies

2. Partitions

A partition is an ordered, immutable sequence of records that is continually appended to. Each partition is replicated across brokers.

Key Points:

- Records within a partition are ordered
- Only one consumer in a consumer group can read from a partition at a time
- Order is guaranteed only within a partition, not across partitions
- Partitions enable parallelism

3. Offset

An offset is a unique identifier for a record within a partition. Offsets are sequential and immutable.

```
Partition 0  
Offset:  0   1   2   3   4   5   6   7  
Record: [msg][msg][msg][msg][msg][msg][msg][msg]  
        └──────────────────────────────────┘  
        Consumer can be at offset 5
```

4. Brokers

Brokers are individual Kafka servers. A cluster consists of multiple brokers.

Responsibilities:

- Store partitions
- Serve read/write requests
- Maintain replicas
- Perform leader election

5. Producer

A producer is an application that publishes records to topics.

Features:

- Sends records to specific topics
- Can specify partition or let Kafka decide
- Can add keys for ordering guarantees
- Configurable delivery semantics

6. Consumer

A consumer is an application that subscribes to topics and processes records.

Features:

- Subscribes to one or more topics
- Reads records in order
- Commits offsets to track progress
- Can be part of a consumer group

7. Consumer Group

A consumer group is a set of consumers that work together to consume a topic.

Key Points:

- Multiple consumers can work in parallel
- Each partition is assigned to one consumer
- If a consumer fails, partitions are rebalanced



- Different consumer groups can read the same topic independently

Topic with 3 Partitions

Consumer Group A:

```
|— Consumer A1 -> Partition 0
|— Consumer A2 -> Partition 1
|— Consumer A3 -> Partition 2
```

Consumer Group B:

```
|— Consumer B1 -> Partition 0, 1
|— Consumer B2 -> Partition 2
```

8. Replication

Each partition is replicated across multiple brokers for fault tolerance.

Components:

- **Leader:** The broker that handles reads and writes for a partition
- **In-Sync Replicas (ISR):** Replicas that are caught up with the leader
- **Follower:** Non-leader replicas

Partition 0 with Replication Factor 3:

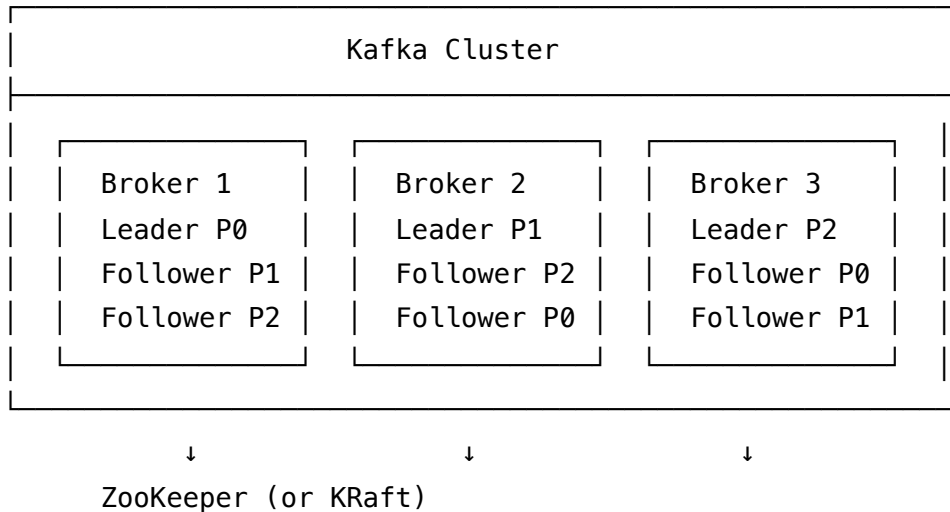
Leader: Broker 1

Followers: Broker 2, Broker 3

Write -> Broker 1 (Leader) -> Brokers 2, 3 (Followers)

Architecture

Kafka Cluster Architecture



Components

Producers

- Send records to topics
- Serialize data before sending
- Optionally partition data using a key
- Handle retries and acknowledgments

Brokers

- Receive records from producers
- Store records on disk
- Serve records to consumers
- Replicate data across the cluster

Consumers

- Read records from topics
- Deserialize data
- Process records
- Commit offsets

ZooKeeper (Deprecated in newer versions)

- Maintains cluster metadata
- Performs broker leader election
- Manages controller election
- In newer versions, replaced by KRaft (Kafka Raft)

KRaft (Kafka Raft)

- New metadata management system (Kafka 3.3+)
- Eliminates dependency on ZooKeeper
- Simpler deployment
- Self-managed metadata quorum

Data Flow

1. Producer sends message to Topic
Producer -> [Serialize] -> Broker (Leader)
2. Broker replicates to followers
Leader -> Replicate -> Follower 1
 \--> Follower 2
3. Producer receives acknowledgment
Broker -> ACK -> Producer
4. Consumer polls for messages
Consumer -> [Poll] -> Broker
5. Consumer processes and commits offset
Consumer -> [Commit Offset] -> Broker

Cluster Setup

Prerequisites

- Java 8 or later (Java 11+ recommended)
- Linux/macOS/Windows



- RAM: Minimum 4GB (8GB+ for production)
- Disk: Based on retention policy

Installation Steps

1. Download Kafka

```
# Download Kafka (latest version)
cd ~
wget https://archive.apache.org/dist/kafka/3.7.0/kafka_2.13-3.7.0.tgz

# Extract
tar -xzf kafka_2.13-3.7.0.tgz
cd kafka_2.13-3.7.0
```

2. Using KRaft (Recommended - No ZooKeeper)

Generate Cluster UUID:

```
KAFKA_CLUSTER_ID="MkQkDryZT20MEYj6metaYg"
```

Configure server.properties:

```
# server.properties
node.id=1
process.roles=broker,controller
controller.quorum.voters=1@localhost:9093
listener.security.protocol.map=CONTROLLER:PLAINTEXT,BROKER:PLAINTEXT,PLAINTEXT:PLAINTEXT
inter.broker.listener.name=BROKER
controller.listener.names=CONTROLLER
advertised.listeners=PLAINTEXT://localhost:9092
log.dirs=/tmp/kraft-logs
```

Format the storage directory:

```
./bin/kafka-storage.sh format -t MkQkDryZT20MEYj6metaYg -c config/kraft/server.properties
```

Start Kafka:



```
./bin/kafka-server-start.sh config/kraft/server.properties
```

3. Using ZooKeeper (Traditional)

Start ZooKeeper:

```
./bin/zookeeper-server-start.sh config/zookeeper.properties
```

Start Kafka Broker:

```
./bin/kafka-server-start.sh config/server.properties
```

4. Multi-Broker Cluster Setup

Broker 1 (config/server-1.properties):

```
broker.id=1  
listeners=PLAINTEXT://localhost:9092  
log.dirs=/tmp/kafka-logs-1
```

Broker 2 (config/server-2.properties):

```
broker.id=2  
listeners=PLAINTEXT://localhost:9093  
log.dirs=/tmp/kafka-logs-2
```

Broker 3 (config/server-3.properties):

```
broker.id=3  
listeners=PLAINTEXT://localhost:9094  
log.dirs=/tmp/kafka-logs-3
```

Start each broker in separate terminal:

```
./bin/kafka-server-start.sh config/server-1.properties  
./bin/kafka-server-start.sh config/server-2.properties  
./bin/kafka-server-start.sh config/server-3.properties
```



Verification

Check running brokers:

```
./bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092
```

List topics:

```
./bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

Developer Setup & Configuration

Java/Kafka Client Dependencies

Maven (pom.xml)

```
<dependency>
  <groupId>org.apache.kafka</groupId>
  <artifactId>kafka-clients</artifactId>
  <version>3.7.0</version>
</dependency>
```

```
<dependency>
  <groupId>org.apache.kafka</groupId>
  <artifactId>kafka-streams</artifactId>
  <version>3.7.0</version>
</dependency>
```

```
<!-- For logging -->
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-simple</artifactId>
  <version>1.7.36</version>
</dependency>
```



Gradle (build.gradle)

```
dependencies {  
    implementation 'org.apache.kafka:kafka-clients:3.7.0'  
    implementation 'org.apache.kafka:kafka-streams:3.7.0'  
    implementation 'org.slf4j:slf4j-simple:1.7.36'  
}
```

Important Configurations

Producer Configurations

Configuration	Default	Description
bootstrap.servers	-	Broker addresses
key.serializer	-	Serializer class for keys
value.serializer	-	Serializer class for values
acks	1	Acknowledgment level
retries	INT_MAX	Number of retries
batch.size	16384	Batch size in bytes
linger.ms	0	Wait time before sending
compression.type	none	Compression type

Consumer Configurations

Configuration	Default	Description
bootstrap.servers	-	Broker addresses
key.deserializer	-	Deserializer class for keys
value.deserializer	-	Deserializer class for values
group.id	-	Consumer group ID
auto.offset.reset	latest	Offset position strategy
enable.auto.commit	true	Auto-commit offsets



Configuration	Default	Description
auto.commit.interval.ms	5000	Commit interval
max.poll.records	500	Max records per poll



Producer API

Basic Producer

```
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.clients.producer.RecordMetadata;
import org.apache.kafka.common.serialization.StringSerializer;
import java.util.Properties;

public class SimpleProducer {
    public static void main(String[] args) {
        // Producer configuration
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);

        // Create producer
        KafkaProducer<String, String> producer = new KafkaProducer<>(props);

        // Send a message
        ProducerRecord<String, String> record = new ProducerRecord<>(
            "my-topic",
            "key1",
            "Hello, Kafka!"
        );

        producer.send(record, (metadata, exception) -> {
            if (exception != null) {
                System.err.println("Error: " + exception.getMessage());
            } else {
                System.out.println("Message sent to partition " +
                    metadata.partition() + " at offset " +
                    metadata.offset());
            }
        });

        producer.close();
    }
}
```



Producer with Callbacks

```
import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.StringSerializer;
import java.util.Properties;

public class ProducerWithCallback {
    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);
        props.put("acks", "all"); // Wait for all replicas
        props.put("retries", 3);
        props.put("compression.type", "snappy");

        KafkaProducer<String, String> producer = new KafkaProducer<>(props);

        for (int i = 0; i < 10; i++) {
            String key = "user-" + i;
            String value = "Event " + i;

            ProducerRecord<String, String> record = new ProducerRecord<>(
                "events-topic",
                key,
                value
            );

            producer.send(record, new Callback() {
                @Override
                public void onCompletion(RecordMetadata metadata, Exception exception)
                    if (exception != null) {
                        System.err.println("Failed to send: " + exception.getMessage())
                    } else {
                        System.out.println(String.format(
                            "Message sent to topic=%s, partition=%d, offset=%d",
                            metadata.topic(),
                            metadata.partition(),
                            metadata.offset()
                        ));
                    }
            });
        }
    }
};
```



```

    }

    producer.close();
}
}

```

Synchronous Producer

```

import org.apache.kafka.clients.producer.*;
import org.apache.kafka.common.serialization.StringSerializer;
import java.util.Properties;

public class SyncProducer {
    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);
        props.put("acks", "all");

        KafkaProducer<String, String> producer = new KafkaProducer<>(props);

        try {
            for (int i = 0; i < 5; i++) {
                ProducerRecord<String, String> record = new ProducerRecord<>(
                    "sync-topic",
                    "key-" + i,
                    "Message " + i
                );

                RecordMetadata metadata = producer.send(record).get();

                System.out.println("Message " + i + " sent to partition " +
                    metadata.partition() + " at offset " + metadata.offset());
            }
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            producer.close();
        }
    }
}

```



Producer with Custom Serializer

```
import org.apache.kafka.common.serialization.Serializer;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.util.Map;

public class User {
    public String id;
    public String name;
    public String email;

    public User(String id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }
}

public class UserSerializer implements Serializer<User> {
    private final ObjectMapper objectMapper = new ObjectMapper();

    @Override
    public void configure(Map<String, ?> configs, boolean isKey) {}

    @Override
    public byte[] serialize(String topic, User data) {
        try {
            return objectMapper.writeValueAsBytes(data);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }

    @Override
    public void close() {}
}

// Usage in Producer
public class UserProducer {
    public static void main(String[] args) throws Exception {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
    }
}
```



```
props.put("key.serializer", StringSerializer.class);
props.put("value.serializer", UserSerializer.class);

KafkaProducer<String, User> producer = new KafkaProducer<>(props);

User user = new User("user-1", "John Doe", "john@example.com");
ProducerRecord<String, User> record = new ProducerRecord<>(
    "users-topic",
    user.id,
    user
);

producer.send(record);
producer.close();
}
}
```

Producer Partitioning Strategies

Custom Partitioner

```
import org.apache.kafka.clients.producer.Partitioner;
import org.apache.kafka.common.Cluster;
import java.util.Map;

public class CustomPartitioner implements Partitioner {
    @Override
    public int partition(String topic, Object key, byte[] keyBytes,
                        Object value, byte[] valueBytes, Cluster cluster) {
        if (key == null) {
            return 0;
        }

        String stringKey = (String) key;

        // Partition based on key prefix
        if (stringKey.startsWith("premium-")) {
            return 0; // Premium messages to partition 0
        } else {
            return 1; // Regular messages to partition 1
        }
    }

    @Override
    public void close() {}

    @Override
    public void configure(Map<String, ?> configs) {}
}

// Usage
public class CustomPartitionerProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);
        props.put("partitioner.class", CustomPartitioner.class);

        KafkaProducer<String, String> producer = new KafkaProducer<>(props);
```



```
producer.send(new ProducerRecord<>("partitioned-topic",  
    "premium-user-1", "High-value customer"));  
producer.send(new ProducerRecord<>("partitioned-topic",  
    "regular-user-1", "Normal customer"));  
  
producer.close();  
}  
}
```



Consumer API

Basic Consumer

```
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.util.Collections;
import java.util.Properties;
import java.time.Duration;

public class SimpleConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "my-consumer-group");
        props.put("key.deserializer", StringDeserializer.class);
        props.put("value.deserializer", StringDeserializer.class);

        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);

        // Subscribe to topic
        consumer.subscribe(Collections.singletonList("my-topic"));

        // Poll for messages
        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(
                Duration.ofMillis(100)
            );

            for (ConsumerRecord<String, String> record : records) {
                System.out.println(String.format(
                    "Received: key=%s, value=%s, partition=%d, offset=%d",
                    record.key(),
                    record.value(),
                    record.partition(),
                    record.offset()
                ));
            }
        }
    }
}
```



}



Consumer with Manual Offset Management

```
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.util.Collections;
import java.util.Properties;
import java.time.Duration;

public class ManualOffsetConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "manual-offset-group");
        props.put("key.deserializer", StringDeserializer.class);
        props.put("value.deserializer", StringDeserializer.class);
        props.put("enable.auto.commit", false); // Disable auto-commit
        props.put("auto.offset.reset", "earliest");

        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList("my-topic"));

        int batchSize = 0;
        int batchSizeLimit = 100;

        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(
                Duration.ofMillis(100)
            );

            for (ConsumerRecord<String, String> record : records) {
                System.out.println(String.format(
                    "Offset: %d, Partition: %d, Value: %s",
                    record.offset(),
                    record.partition(),
                    record.value()
                ));

                batchSize++;

                // Commit periodically
                if (batchSize >= batchSizeLimit) {
                    consumer.commitSync();
                    System.out.println("Offsets committed");
                }
            }
        }
    }
}
```



```
        batchSize = 0;
    }
}
}
```



Consumer with Rebalance Listener

```
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.TopicPartition;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.util.*;
import java.time.Duration;

public class RebalanceListenerConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "rebalance-group");
        props.put("key.deserializer", StringDeserializer.class);
        props.put("value.deserializer", StringDeserializer.class);
        props.put("enable.auto.commit", false);

        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);

        // Add rebalance listener
        ConsumerRebalanceListener rebalanceListener =
            new ConsumerRebalanceListener() {
                @Override
                public void onPartitionsRevoked(Collection<TopicPartition> partitions)
                    System.out.println("Partitions revoked: " + partitions);
                    consumer.commitSync(); // Commit before losing partitions
                }

                @Override
                public void onPartitionsAssigned(Collection<TopicPartition> partitions)
                    System.out.println("Partitions assigned: " + partitions);
                }
            };

        consumer.subscribe(Collections.singletonList("my-topic"),
            rebalanceListener);

        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(
                Duration.ofMillis(100)
            );

            for (ConsumerRecord<String, String> record : records) {
```



```
        System.out.println("Value: " + record.value());  
    }  
}  
}
```



Consumer with Custom Deserializer

```
import org.apache.kafka.common.serialization.Deserializer;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.util.Map;

public class User {
    public String id;
    public String name;
    public String email;
}

public class UserDeserializer implements Deserializer<User> {
    private final ObjectMapper objectMapper = new ObjectMapper();

    @Override
    public void configure(Map<String, ?> configs, boolean isKey) {}

    @Override
    public User deserialize(String topic, byte[] data) {
        try {
            return objectMapper.readValue(data, User.class);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }

    @Override
    public void close() {}
}

// Usage
public class UserConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "user-consumer-group");
        props.put("key.deserializer", StringDeserializer.class);
        props.put("value.deserializer", UserDeserializer.class);

        KafkaConsumer<String, User> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList("users-topic"));
    }
}
```

```
while (true) {  
    ConsumerRecords<String, User> records = consumer.poll(  
        Duration.ofMillis(100)  
    );  
  
    for (ConsumerRecord<String, User> record : records) {  
        User user = record.value();  
        System.out.println("User: " + user.name + " (" + user.email + ")");  
    }  
}  
}
```

Advanced Topics

Exactly Once Semantics (EOS)

Exactly once delivery is critical for transactional systems.



Producer Side - Idempotent Producer

```
public class IdempotentProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);
        props.put("enable.idempotence", true); // Enable idempotence
        props.put("acks", "all");
        props.put("retries", Integer.MAX_VALUE);
        props.put("max.in.flight.requests.per.connection", 5);

        KafkaProducer<String, String> producer = new KafkaProducer<>(props);

        for (int i = 0; i < 100; i++) {
            ProducerRecord<String, String> record = new ProducerRecord<>(
                "idempotent-topic",
                "key-" + i,
                "Message " + i
            );
            producer.send(record);
        }

        producer.close();
    }
}
```



Transactional Producer

```
public class TransactionalProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);
        props.put("transactional.id", "unique-transaction-id");
        props.put("enable.idempotence", true);
        props.put("acks", "all");

        KafkaProducer<String, String> producer = new KafkaProducer<>(props);

        producer.initTransactions();

        try {
            producer.beginTransaction();

            for (int i = 0; i < 10; i++) {
                ProducerRecord<String, String> record = new ProducerRecord<>(
                    "transactional-topic",
                    "key-" + i,
                    "Transactional message " + i
                );
                producer.send(record);
            }

            producer.commitTransaction();
            System.out.println("Transaction committed successfully");
        } catch (Exception e) {
            producer.abortTransaction();
            System.out.println("Transaction aborted due to: " + e.getMessage());
        } finally {
            producer.close();
        }
    }
}
```



Transactional Consumer

```
public class TransactionalConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "transactional-group");
        props.put("key.deserializer", StringDeserializer.class);
        props.put("value.deserializer", StringDeserializer.class);
        props.put("isolation.level", "read_committed"); // Read only committed message
        props.put("enable.auto.commit", false);

        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList("transactional-topic"));

        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(
                Duration.ofMillis(100)
            );

            for (ConsumerRecord<String, String> record : records) {
                System.out.println("Read committed message: " + record.value());
            }

            consumer.commitSync();
        }
    }
}
```

Kafka Streams

Kafka Streams allows processing of streaming data in real-time.



Basic Stream Application

```
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.KafkaStreams;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.kstream.KStream;
import java.util.Properties;

public class WordCountApp {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("application.id", "wordcount-app");
        props.put("bootstrap.servers", "localhost:9092");
        props.put("default.key.serde", Serdes.String().getClass());
        props.put("default.value.serde", Serdes.String().getClass());

        StreamsBuilder builder = new StreamsBuilder();

        KStream<String, String> source = builder.stream("input-topic");

        source
            .flatMapValues(value -> Arrays.asList(value.split("\\s+")))
            .map((key, word) -> new KeyValue<>(word, word))
            .groupByKey()
            .count()
            .toStream()
            .to("output-topic");

        KafkaStreams streams = new KafkaStreams(builder.build(), props);
        streams.start();
    }
}
```

Stateful Operations (Aggregation)

```
public class AggregationApp {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("application.id", "aggregation-app");
        props.put("bootstrap.servers", "localhost:9092");
        props.put("default.key.serde", Serdes.String().getClass());
        props.put("default.value.serde", Serdes.String().getClass());

        StreamsBuilder builder = new StreamsBuilder();

        KStream<String, String> source = builder.stream("sales-topic");

        source
            .mapValues(value -> {
                try {
                    return Integer.parseInt(value);
                } catch (Exception e) {
                    return 0;
                }
            })
            .groupByKey()
            .aggregate(
                () -> 0, // Initial value
                (key, value, aggregate) -> aggregate + value // Aggregator
            )
            .toStream()
            .to("sales-total-topic");

        KafkaStreams streams = new KafkaStreams(builder.build(), props);
        streams.start();
    }
}
```

Windowing Operations

```
import org.apache.kafka.streams.kstream.TimeWindows;
import java.time.Duration;

public class WindowingApp {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("application.id", "windowing-app");
        props.put("bootstrap.servers", "localhost:9092");
        props.put("default.key.serde", Serdes.String().getClass());
        props.put("default.value.serde", Serdes.String().getClass());

        StreamsBuilder builder = new StreamsBuilder();

        KStream<String, String> source = builder.stream("events-topic");

        source
            .groupByKey()
            .windowedBy(TimeWindows.of(Duration.ofSeconds(10)))
            .count()
            .toStream()
            .map((key, value) -> KeyValue.pair(
                key.key() + "@" + key.window().startTime(),
                value
            ))
            .to("windowed-output-topic");

        KafkaStreams streams = new KafkaStreams(builder.build(), props);
        streams.start();
    }
}
```

Schema Registry Integration

Schema Registry helps manage data schemas centrally.



Avro Schema Definition

```
{
  "type": "record",
  "name": "User",
  "namespace": "com.example.kafka",
  "fields": [
    {"name": "id", "type": "string"},
    {"name": "name", "type": "string"},
    {"name": "email", "type": "string"},
    {"name": "age", "type": "int"}
  ]
}
```

Producer with Schema Registry

```
<!-- pom.xml -->
<dependency>
  <groupId>io.confluent</groupId>
  <artifactId>kafka-avro-serializer</artifactId>
  <version>7.5.0</version>
</dependency>
<dependency>
  <groupId>org.apache.avro</groupId>
  <artifactId>avro</artifactId>
  <version>1.11.0</version>
</dependency>
```



```

import io.confluent.kafka.serializers.KafkaAvroSerializer;
import com.example.kafka.User;

public class AvroProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", KafkaAvroSerializer.class);
        props.put("schema.registry.url", "http://localhost:8081");

        KafkaProducer<String, User> producer = new KafkaProducer<>(props);

        User user = new User("user-1", "John Doe", "john@example.com", 30);

        ProducerRecord<String, User> record = new ProducerRecord<>(
            "users-avro-topic",
            user.getId().toString(),
            user
        );

        producer.send(record);
        producer.close();
    }
}

```



Consumer with Schema Registry

```
import io.confluent.kafka.serializers.KafkaAvroDeserializer;
import com.example.kafka.User;

public class AvroConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "avro-consumer-group");
        props.put("key.deserializer", StringDeserializer.class);
        props.put("value.deserializer", KafkaAvroDeserializer.class);
        props.put("schema.registry.url", "http://localhost:8081");
        props.put("specific.avro.reader", true);

        KafkaConsumer<String, User> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList("users-avro-topic"));

        while (true) {
            ConsumerRecords<String, User> records = consumer.poll(
                Duration.ofMillis(100)
            );

            for (ConsumerRecord<String, User> record : records) {
                User user = record.value();
                System.out.println("User: " + user.getName() +
                    ", Age: " + user.getAge());
            }
        }
    }
}
```

Change Data Capture (CDC)

Capturing changes from databases.



```

import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.util.Collections;
import java.util.Properties;
import java.time.Duration;

public class CDCConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "cdc-consumer");
        props.put("key.deserializer", StringDeserializer.class);
        props.put("value.deserializer", StringDeserializer.class);
        props.put("isolation.level", "read_committed");

        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList("db-changes"));

        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(
                Duration.ofMillis(100)
            );

            for (ConsumerRecord<String, String> record : records) {
                // Parse CDC event (Debezium format typically)
                String operation = extractOperation(record.value());
                String table = extractTable(record.value());
                String data = extractData(record.value());

                System.out.println(String.format(
                    "Operation: %s, Table: %s, Data: %s",
                    operation, table, data
                ));

                // Process the change
                processDBChange(operation, table, data);
            }
        }

        private static String extractOperation(String json) {
            // Parse JSON to extract operation type
            return "INSERT"; // Placeholder
        }
    }
}

```



```

}

private static String extractTable(String json) {
    return "users"; // Placeholder
}

private static String extractData(String json) {
    return json; // Placeholder
}

private static void processDBChange(String op, String table, String data) {
    // Handle database changes
}
}

```

Best Practices

1. Partitioning Strategy

Use consistent keys:

```

// Good – same user always goes to same partition
producer.send(new ProducerRecord<>(
    "events-topic",
    user.getId(), // Use consistent key
    eventData
));

// Avoid – different keys for same entity
producer.send(new ProducerRecord<>(
    "events-topic",
    UUID.randomUUID().toString(), // Non-deterministic
    eventData
));

```



2. Error Handling

```
public class RobustProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);
        props.put("acks", "all");
        props.put("retries", 3);
        props.put("retry.backoff.ms", 100);

        KafkaProducer<String, String> producer = new KafkaProducer<>(props);

        ProducerRecord<String, String> record = new ProducerRecord<>(
            "resilient-topic",
            "key-1",
            "message"
        );

        producer.send(record, (metadata, exception) -> {
            if (exception != null) {
                // Log error
                System.err.println("Send failed: " + exception.getMessage());

                // Implement retry logic or dead letter queue
                sendToDeadLetterQueue(record, exception);
            } else {
                System.out.println("Message sent successfully");
            }
        });

        producer.close();
    }

    private static void sendToDeadLetterQueue(ProducerRecord record,
        Exception exception) {
        // Send to DLQ topic
    }
}
```

3. Consumer Lag Monitoring

```
import org.apache.kafka.clients.consumer.OffsetAndMetadata;
import org.apache.kafka.common.TopicPartition;

public class ConsumerLagMonitor {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "lag-monitor-group");
        props.put("key.deserializer", StringDeserializer.class);
        props.put("value.deserializer", StringDeserializer.class);

        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList("monitored-topic"));

        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(
                Duration.ofMillis(100)
            );

            if (records.isEmpty()) {
                // Calculate lag
                Set<TopicPartition> partitions = consumer.assignment();

                for (TopicPartition partition : partitions) {
                    long highwaterMark = consumer.position(partition);
                    OffsetAndMetadata offsetMetadata = consumer.committed(partition);
                    long committedOffset = offsetMetadata != null ?
                        offsetMetadata.offset() : 0;

                    long lag = highwaterMark - committedOffset;
                    System.out.println("Lag for " + partition + ": " + lag);
                }
            }

            for (ConsumerRecord<String, String> record : records) {
                // Process record
            }
        }
    }
}
```



4. Resource Management

```
public class ResourceManagement {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);
        props.put("connections.max.idle.ms", 540000);
        props.put("socket.keepalive.enabled", true);

        KafkaProducer<String, String> producer = new KafkaProducer<>(props);

        try {
            // Use producer
            producer.send(new ProducerRecord<>(
                "resource-topic",
                "key",
                "value"
            ));
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            // Always close resources
            producer.flush(); // Wait for pending sends
            producer.close(); // Close producer
        }
    }
}
```

5. Consumer Configuration Best Practices

```
public class OptimizedConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("group.id", "optimized-group");
        props.put("key.deserializer", StringDeserializer.class);
        props.put("value.deserializer", StringDeserializer.class);

        // Performance tuning
        props.put("fetch.min.bytes", 1024);           // Min bytes to fetch
        props.put("fetch.max.wait.ms", 500);          // Max wait time
        props.put("max.poll.records", 500);           // Records per poll
        props.put("max.poll.interval.ms", 300000);    // Max poll interval

        // Offset management
        props.put("enable.auto.commit", false);       // Manual commits
        props.put("auto.offset.reset", "earliest");

        // Session management
        props.put("session.timeout.ms", 10000);
        props.put("heartbeat.interval.ms", 3000);

        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList("optimized-topic"));

        while (true) {
            ConsumerRecords<String, String> records = consumer.poll(
                Duration.ofMillis(100)
            );

            for (ConsumerRecord<String, String> record : records) {
                // Process record
            }

            // Commit after processing
            if (!records.isEmpty()) {
                consumer.commitSync();
            }
        }
    }
}
```

```
}  
}
```

6. Topic Design

```
# Create topic with optimal configuration  
./bin/kafka-topics.sh --create \  
  --bootstrap-server localhost:9092 \  
  --topic optimized-topic \  
  --partitions 12 \  
  --replication-factor 3 \  
  --config retention.ms=604800000 \  
  --config compression.type=snappy \  
  --config min.insync.replicas=2
```



7. Producer Configuration Best Practices

```
public class OptimizedProducer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);

        // Reliability
        props.put("acks", "all");                // Wait for all replicas
        props.put("min.insync.replicas", 2);      // Min in-sync replicas
        props.put("retries", Integer.MAX_VALUE);  // Unlimited retries
        props.put("enable.idempotence", true);    // Idempotent writes

        // Performance
        props.put("batch.size", 32768);           // 32 KB batches
        props.put("linger.ms", 10);              // Wait 10ms for batching
        props.put("compression.type", "snappy");  // Compression
        props.put("buffer.memory", 67108864);    // 64 MB buffer

        // Network
        props.put("socket.keepalive.enabled", true);
        props.put("connections.max.idle.ms", 540000);

        KafkaProducer<String, String> producer = new KafkaProducer<>(props);

        producer.send(new ProducerRecord<>("perf-topic", "key", "value"));
        producer.close();
    }
}
```

Troubleshooting

Common Issues and Solutions

1. Consumer Lag Increasing

```
# Check consumer group status
./bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group my-group --describe
```

```
# Output shows lag per partition
```

TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG
my-topic	0	100	500	400
my-topic	1	150	600	450

Solution:

- Increase number of consumers or partitions
- Optimize message processing logic
- Increase consumer performance settings

2. UnknownTopicOrPartition Error

```
// Error: org.apache.kafka.common.errors.UnknownTopicOrPartition
```

```
// Solution: Check if topic exists
```

```
./bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

```
// Create topic if missing
```

```
./bin/kafka-topics.sh --create --bootstrap-server localhost:9092 \
  --topic missing-topic --partitions 3 --replication-factor 1
```

3. NotLeaderForPartition Error

```
# Check partition leaders
```

```
./bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --topic my-topic --describe
```

```
# Solution: Restart the broker acting as leader
```

```
# or perform leader election
```



4. Offset Out of Range

```
// Error: org.apache.kafka.common.errors.OffsetOutOfRange

// Solution: Reset consumer offset
./bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group my-group --topic my-topic --reset-offsets \
  --to-earliest --execute

// Or to specific offset
./bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group my-group --topic my-topic --reset-offsets \
  --to-offset 1000 --execute
```

5. Broker Connections Failing

```
// Check broker connectivity
import java.net.Socket;

public class BrokerConnectivityTest {
    public static void main(String[] args) {
        String brokerHost = "localhost";
        int brokerPort = 9092;

        try {
            Socket socket = new Socket(brokerHost, brokerPort);
            System.out.println("Successfully connected to broker");
            socket.close();
        } catch (Exception e) {
            System.err.println("Failed to connect: " + e.getMessage());
        }
    }
}
```

6. Message Loss Debugging

```
public class MessageLossDebug {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("bootstrap.servers", "localhost:9092");
        props.put("key.serializer", StringSerializer.class);
        props.put("value.serializer", StringSerializer.class);

        // Enable logging
        props.put("loglevel", "debug");

        // Producer configuration for zero message loss
        props.put("acks", "all");
        props.put("retries", Integer.MAX_VALUE);
        props.put("max.in.flight.requests.per.connection", 5);

        KafkaProducer<String, String> producer = new KafkaProducer<>(props);

        for (int i = 0; i < 1000; i++) {
            ProducerRecord<String, String> record = new ProducerRecord<>(
                "debug-topic",
                String.valueOf(i),
                "Message " + i
            );

            try {
                RecordMetadata metadata = producer.send(record).get();
                System.out.println("Sent message " + i + " to offset " +
                    metadata.offset());
            } catch (Exception e) {
                System.err.println("Failed to send message " + i + ": " +
                    e.getMessage());
            }
        }

        producer.close();
    }
}
```

7. Consumer Rebalancing Issues

```
# Check for frequent rebalancing
# Look for RebalanceInProgress errors in logs

# Solutions:
# 1. Increase max.poll.interval.ms in consumer config
# 2. Increase session.timeout.ms
# 3. Optimize processing logic to prevent timeouts
# 4. Increase max.poll.records if processing is slow

# Monitor rebalancing
./bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --group my-group --describe

# Check logs
tail -f /path/to/kafka/logs/server.log | grep -i rebalance
```

8. High CPU Usage

```
# Check producer/consumer load
# Profile the application
# Optimize:
# - Serialization logic
# - Deserialization logic
# - Message processing logic
# - Compression/decompression

# Reduce compression.type or increase batch.size to spread overhead
```

9. Memory Issues

```
# Increase JVM heap size
export KAFKA_HEAP_OPTS="-Xmx2G -Xms2G"

# For producer/consumer applications
export KAFKA_CLIENT_HEAP_OPTS="-Xmx1G -Xms1G"
```



10. Network Issues

```
# Check network connectivity  
ping broker-host
```

```
# Check port accessibility  
nc -zv broker-host 9092
```

```
# Verify firewall rules  
# Check DNS resolution  
nslookup broker-host
```



Advanced Configuration

Kafka Broker Configuration

```
# server.properties – Critical settings

# Broker identification
broker.id=1
node.id=1

# Network configuration
listeners=PLAINTEXT://0.0.0.0:9092,SSL://0.0.0.0:9093,SASL_SSL://0.0.0.0:9094
advertised.listeners=PLAINTEXT://kafka-broker-1.example.com:9092
listener.security.protocol.map=PLAINTEXT:PLAINTEXT,SSL:SSL,SASL_SSL:SASL_SSL

# Storage
log.dirs=/var/kafka/logs
log.retention.hours=168
log.segment.bytes=1073741824
log.retention.check.interval.ms=300000

# Replication
min.insync.replicas=2
default.replication.factor=3

# Performance
num.network.threads=8
num.io.threads=8
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600

# Cleanup policy
log.cleanup.policy=delete
log.cleanup.delete.retention.ms=604800000

# Compression
compression.type=snappy

# Group Coordinator
```



```
offsets.topic.replication.factor=3
transaction.state.log.replication.factor=3
```

SSL/TLS Configuration

```
# Producer
security.protocol=SSL
ssl.truststore.location=/path/to/truststore.jks
ssl.truststore.password=password
ssl.keystore.location=/path/to/keystore.jks
ssl.keystore.password=password
ssl.key.password=password

# Consumer
security.protocol=SSL
ssl.truststore.location=/path/to/truststore.jks
ssl.truststore.password=password
```

SASL Configuration

```
# SASL/PLAINTEXT
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule \
    required username="user" password="password";

# SASL/SSL
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
ssl.truststore.location=/path/to/truststore.jks
ssl.truststore.password=password
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule \
    required username="user" password="password";
```



Monitoring and Observability

JMX Metrics

```
import javax.management.MBeanServer;
import java.lang.management.ManagementFactory;

public class MetricsMonitor {
    public static void main(String[] args) throws Exception {
        MBeanServer mbs = ManagementFactory.getPlatformMBeanServer();

        // Query JMX beans for Kafka metrics
        // Producer metrics
        // Consumer metrics
        // Broker metrics
    }
}
```

Common Metrics to Monitor

Metric	Description	Threshold
Consumer Lag	Offset difference	Monitor trend
Producer Rate	Messages/sec	Expected range
Consumer Rate	Messages/sec	Expected range
Broker Disk Usage	Percentage used	< 80%
Broker Memory Usage	Percentage used	< 85%
Network I/O	Bytes in/out	Monitor trend
Request Latency	Average latency	< 100ms

Conclusion

This guide covers the essential aspects of Apache Kafka for developers. Key takeaways:



1. **Understand the fundamentals:** Topics, partitions, offsets, and replication
2. **Design for scale:** Proper partitioning and configuration
3. **Implement reliability:** Acknowledgments, idempotence, and transactions
4. **Monitor continuously:** Track consumer lag, throughput, and latency
5. **Handle failures:** Plan for broker failures and network issues
6. **Optimize performance:** Tune batch sizes, compression, and threading

For production systems, always:

- Test thoroughly before deployment
- Monitor key metrics continuously
- Have runbooks for common issues
- Plan for disaster recovery
- Document your configurations and topology

Additional Resources

- **Official Documentation:** <https://kafka.apache.org/documentation/>
- **Confluent Documentation:** <https://docs.confluent.io/>
- **GitHub:** <https://github.com/apache/kafka>
- **Community:** <https://kafka.apache.org/community>

Last Updated: December 2024

Kafka Version: 3.7.0

